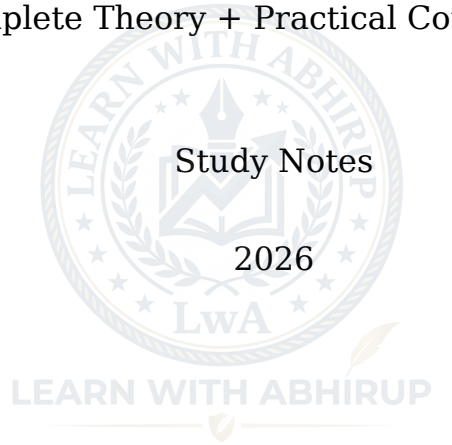
Artificial Intelligence & Machine Learning

A Complete Theory + Practical Course (A to Z)



Study Notes

2026

Contents

Chapter 1

Preface

This course covers **Artificial Intelligence and Machine Learning** end-to-end — every module builds on the last:

1. **Module 1** — What AI is, its history, how knowledge is represented, and how machines search for solutions.
2. **Module 2** — The formal logic that lets machines reason and draw conclusions.
3. **Module 3** — What Machine Learning is, its three major paradigms, and the statistics underneath it.
4. **Module 4** — The two core supervised/unsupervised algorithm families: trees & regression, and clustering.
5. **Module 5** — Neural networks (the basis of deep learning) and genetic algorithms (evolution-inspired optimization).

How to use this book: - Each topic follows the same pattern: **theory → why it matters → a real-life example → (where applicable) runnable Python code with expected output.** - All code has been tested and runs as-is with standard libraries (`numpy`, `pandas`, `scikit-learn`). - Read a section, then actually run its code — modify the numbers, break it, fix it. That's how the concepts become intuition instead of memorized definitions.

Chapter 2

Module 1: Concepts of AI and Searching Techniques

2.1 1.1 What is Artificial Intelligence?

Artificial Intelligence (AI) is the branch of computer science concerned with building machines that can perform tasks which normally require human intelligence — understanding language, recognizing images, making decisions, planning, and learning from experience.

> **Simple definition:** AI = making a computer *behave* in ways we would call "intelligent" if a human did them.

Real-life anchor: When you unlock your phone with your face, when Google Maps reroutes you around traffic, when Gmail suggests how to finish your sentence, when Netflix recommends a show you actually like — all of these are AI systems quietly working in the background. AI is not a single algorithm; it is a *goal* (intelligent behavior) that many different techniques (search, logic, statistics, neural networks) try to achieve.

2.1.1 Why AI matters today

- **Healthcare:** AI reads X-rays/MRIs faster and sometimes more accurately than junior radiologists.
- **Finance:** Fraud-detection systems flag suspicious card transactions in milliseconds.
- **Transportation:** Self-driving cars perceive the road and plan a path in real time.
- **Everyday life:** Voice assistants (Siri, Alexa), spam filters, autocomplete, translation apps.

2.2 1.2 History and Evolution of AI

Year	Milestone	Why it mattered
1950	Alan Turing publishes *Computing Machinery and Intelligence*, proposes the **Turing Test**	First rigorous way to ask "can machines think?"
1956	**Dartmouth Conference** (McCarthy, Minsky, Rochester, Shannon)	The term "Artificial Intelligence" is coined; the field is officially born
1966	**ELIZA** (Weizenbaum)	Pattern-matching chatbot simulating a therapist — showed machines can *appear* to converse
1970s	**SHRDLU** (Winograd)	Understood natural-language commands in a virtual "blocks world"
1980s	**Expert Systems boom** (MYCIN, DENDRAL)	Rule-based systems mimicked human experts in narrow domains
1987–93	**Second AI Winter**	Expert systems proved brittle and expensive; funding collapsed
1997	**Deep Blue beats Kasparov**	Brute-force search + domain knowledge beats a world chess champion
2012	**AlexNet** wins ImageNet	Deep convolutional neural networks prove dramatically superior at vision tasks
2016	**AlphaGo beats Lee Sedol**	Deep reinforcement learning masters Go, once thought "too intuitive" for machines
2018–2023	**Large Language Models** (BERT, GPT)	Machines generate fluent, context-aware text at scale

2.2.1 AI Winters

An **AI Winter** is a period when hype outran results, funding dried up, and progress stalled.

- **1974–1980:** Early promises (machine translation, general problem solvers) failed to scale to real-world complexity.
- **Late 1980s–early 1990s:** Expert systems were expensive to maintain and could not learn or adapt.

Lesson for practitioners: keep expectations grounded in what data, hardware, and

algorithms can *actually* deliver — a lesson still relevant when evaluating today's AI hype cycles.

2.3 1.3 Foundations of AI (Interdisciplinary Roots)

AI does not stand alone — it borrows from many older fields.

Field	Contribution	Example
Computer Science	Algorithms, data structures, computing power	GPUs make deep learning practical
Mathematics	Logic, probability, linear algebra, calculus	Backpropagation is just calculus (chain rule)
Psychology / Cognitive Science	Models of how humans think, learn, remember	Inspired reinforcement learning ("reward" mimics behaviorism)
Neuroscience	Structure of the brain	Neural networks are loosely inspired by biological neurons
Linguistics	Structure of language (grammar, semantics)	Powers machine translation, chatbots
Philosophy (Ethics/Logic)	Formal reasoning; questions of mind and morality	AI ethics, bias, and fairness debates

2.4 1.4 Types of AI

2.4.1 By capability

1. **Narrow AI (Weak AI):** Designed for one specific task. *Example: a spam filter, a chess engine, a face-unlock system.* Almost all AI in use today is narrow AI.
2. **General AI (Strong AI):** A hypothetical machine with human-level intelligence across *any* task. Does not exist yet.
3. **Superintelligence:** A hypothetical intelligence surpassing the best human minds in every field. Purely theoretical/futuristic.

2.4.2 By functionality

1. **Reactive Machines:** No memory; react only to current input. *Example: Deep Blue — it evaluated the current board, with no memory of past games.*
2. **Limited Memory:** Uses recent past data to make decisions. *Example: a self-driving car uses the last few seconds of sensor data to judge the speed of nearby cars.*
3. **Theory of Mind (research stage):** Would understand emotions, beliefs, and intentions of others.

4. **Self-Aware AI (hypothetical):** Would have its own consciousness. Science fiction — not real today.

2.5 1.5 Knowledge Representation

For a machine to "reason," its knowledge must be represented in a form it can manipulate.

2.5.1 Declarative Knowledge — *"knowing that"*

Facts and relationships, stated directly. - **Semantic networks:** nodes = concepts, edges = relationships. *Example: "Canary → is-a → Bird → is-a → Animal."* - **Frames:** structured templates with slots. *Example: a "Restaurant" frame has slots for Waiter, Menu, Customer — this is literally how many chatbot/booking systems structure information today.*

2.5.2 Procedural Knowledge — *"knowing how"*

Knowledge of *how* to do something, encoded as rules or algorithms. - **Production rules:** `IF temperature > 100°C THEN turn off heater.` *(This is exactly how a thermostat or industrial safety controller works.)* - **Scripts:** a stereotyped sequence of events. *Example: Restaurant script → Enter → Order → Eat → Pay → Leave — used in older NLP systems to understand stories.* - **Algorithms:** sorting, searching, path-finding.

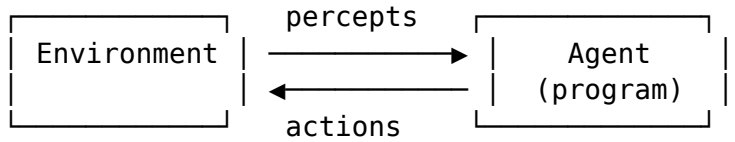
2.5.3 Meta-Knowledge — *"knowing what you know"*

Knowledge about knowledge — e.g., a medical expert system knowing *which* of its rules are most reliable, or *when* to say "I don't know" and refer to a specialist.

Real-life tie-together: A hospital triage chatbot might use declarative knowledge (symptom → disease facts), procedural knowledge (rules for what question to ask next), and meta-knowledge (knowing when a case is too complex and must go to a human doctor).

2.6 1.6 Intelligent Agents

An **agent** is anything that *perceives* its environment through sensors and *acts* upon it through actuators.



2.6.1 The Agent Cycle

1. **Perceiving the environment** — sensors collect data (camera, microphone, GPS).
2. **Processing information & deciding** — the agent's "brain" reasons about what to do.
3. **Taking action** — actuators carry out the decision (motors, screen output, speakers).
4. **Learning & improving over time** — the agent updates its behavior based on outcomes.

2.6.2 PEAS Framework (used to *specify* an agent)

Element	Meaning	Example — Self-driving car
Performance measure	What counts as success	Safety, speed, legality, comfort
Environment	The world the agent acts in	Roads, traffic, pedestrians, weather
Actuators	How it acts	Steering, brakes, accelerator, signals
Sensors	How it perceives	Cameras, LIDAR, GPS, speedometer

2.6.3 Types of Agents (increasing sophistication)

1. **Simple Reflex Agent** — acts only on the current percept via condition-action rules. *Example: a thermostat.*
2. **Model-Based Reflex Agent** — keeps an internal model of the world to handle partial observability. *Example: a robot vacuum that remembers which rooms it already cleaned.*
3. **Goal-Based Agent** — acts to achieve a defined goal, considering future consequences. *Example: a GPS navigator finding a route to a destination.*
4. **Utility-Based Agent** — chooses actions that maximize a "happiness"/utility score, not just any path to the goal. *Example: the GPS navigator now also weighs fastest **vs** most fuel-efficient **vs** most scenic route.*
5. **Learning Agent** — improves its performance over time using feedback. *Example: Netflix's recommendation engine gets better as you rate more shows.*

2.7 1.7 Problem Solving as Search

Many AI problems (route-finding, puzzle-solving, game-playing) can be reframed as a **search problem**:

- **State:** a configuration of the world (e.g., current city).
- **Initial state:** where we start.

- **Goal state / test:** how we know we've arrived.
- **Actions / successor function:** what moves are possible from a state.
- **Path cost:** the cost of a sequence of actions (distance, time, money).

Real-life example: Google Maps treats every intersection as a *state*, every road segment as an *action* with a cost (distance/time), and finds a path from your current state to your destination state.

2.8 1.8 Uninformed (Blind) Search Algorithms

These algorithms have **no extra knowledge** about how far a state is from the goal — they only know the problem definition.

2.8.1 1.8.1 Breadth-First Search (BFS)

Explores the search tree **level by level** using a FIFO queue. Guarantees the *shortest path* (fewest edges) if all step costs are equal.

Algorithm:

```
1. Put the start node in a queue.
2. While queue is not empty:
   a. Dequeue a node.
   b. If it is the goal, return the path.
   c. Otherwise, enqueue all its unvisited neighbors.
```

Real-life example: Finding the shortest number of hops between two people on a social network ("degrees of separation") — LinkedIn's "how you're connected" feature uses BFS-like logic.

2.8.2 1.8.2 Depth-First Search (DFS)

Explores as far as possible down one branch before backtracking, using a LIFO stack (or recursion).

Real-life example: Solving a maze by always trying to go forward and only backing up when you hit a dead end.

2.8.3 1.8.3 Uniform-Cost Search (UCS)

Like BFS, but always expands the node with the **lowest cumulative path cost** (uses a priority queue). Guarantees the optimal path even when costs differ.

Real-life example: Google Maps choosing the cheapest-toll route, where each road segment has a different cost, not just "1 hop."

2.8.4 1.8.4 Depth-Limited Search (DLS) and Iterative Deepening DFS (IDDFS)

- **DLS:** DFS with a maximum depth cutoff — prevents infinite loops in infinite state spaces.
- **IDDFS:** Repeats DLS with increasing depth limits (0, 1, 2, …) until the goal is found. Combines DFS's low memory use with BFS's completeness and optimality (for uniform costs).

2.9 1.9 Informed (Heuristic) Search Algorithms

These use a **heuristic function h(n)** — an *estimate* of the cost from node `n` to the goal — to search more intelligently.

Real-life example of a heuristic: the straight-line ("as the crow flies") distance between your location and your destination is a heuristic for the actual road distance — it's cheap to compute and usually a good guess.

2.9.1 1.9.1 Greedy Best-First Search

Always expands the node that *looks* closest to the goal, i.e., minimizes `h(n)`. Fast, but not guaranteed optimal — it can be "fooled" by a good-looking dead end.

2.9.2 1.9.2 A* Search

Combines the cost so far and the estimated cost to go:

$$f(n) = g(n) + h(n)$$

where `g(n)` = actual cost from start to `n`, and `h(n)` = estimated cost from `n` to goal.

A* is **optimal and complete** as long as `h(n)` never overestimates the true cost (an "admissible" heuristic).

Real-life example: A* is the algorithm behind pathfinding in almost every video game (NPCs finding their way around obstacles) and is a core building block inside real routing engines like Google Maps and GPS units.

2.10 1.10 PRACTICAL: Implementing Search Algorithms in Python

We will model a small map of cities (a classic AI textbook example) as a graph and find routes between cities using BFS, DFS, UCS, and A*.

```python
# graph_search.py
# A small "map" of cities with road distances (like a mini Google Maps)
import heapq

# Adjacency list: city -> list of (neighbor, distance_in_km)
graph = {
    'Kolkata':   [('Durgapur', 170), ('Asansol', 200)],
    'Durgapur':  [('Kolkata', 170), ('Asansol', 40), ('Bardhaman', 50)],
    'Asansol':   [('Kolkata', 200), ('Durgapur', 40), ('Dhanbad', 60)],
    'Bardhaman': [('Durgapur', 50), ('Kolkata', 100)],
    'Dhanbad':   [('Asansol', 60), ('Ranchi', 160)],
    'Ranchi':    [('Dhanbad', 160)]
}

# Straight-line heuristic estimates to the goal city "Ranchi" (in km,
 ↪  approximate)
heuristic_to_ranchi = {
    'Kolkata': 380, 'Durgapur': 260, 'Asansol': 210,
    'Bardhaman': 300, 'Dhanbad': 150, 'Ranchi': 0
}


def bfs(start, goal):
    """Breadth-First Search: fewest hops, ignores distance."""
    frontier = [[start]]
    visited = {start}
    while frontier:
        path = frontier.pop(0)
        node = path[-1]
        if node == goal:
            return path
        for neighbor, _ in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                frontier.append(path + [neighbor])
    return None


def dfs(start, goal, path=None, visited=None):
    """Depth-First Search: goes deep first, backtracks on dead ends."""
    if path is None:
        path, visited = [start], {start}
    if start == goal:
        return path
    for neighbor, _ in graph.get(start, []):
        if neighbor not in visited:
            visited.add(neighbor)
```

```
            result = dfs(neighbor, goal, path + [neighbor], visited)
            if result:
                return result
    return None


def uniform_cost_search(start, goal):
    """UCS: always expand the cheapest total-cost path so far."""
    frontier = [(0, [start])]  # (cumulative_cost, path)
    visited = set()
    while frontier:
        cost, path = heapq.heappop(frontier)
        node = path[-1]
        if node == goal:
            return path, cost
        if node in visited:
            continue
        visited.add(node)
        for neighbor, weight in graph.get(node, []):
            if neighbor not in visited:
                heapq.heappush(frontier, (cost + weight, path + [neighbor]))
    return None, float('inf')


def a_star_search(start, goal, heuristic):
    """A*: cost-so-far (g) + estimated-cost-to-goal (h)."""
    frontier = [(heuristic[start], 0, [start])]  # (f, g, path)
    visited = set()
    while frontier:
        f, g, path = heapq.heappop(frontier)
        node = path[-1]
        if node == goal:
            return path, g
        if node in visited:
            continue
        visited.add(node)
        for neighbor, weight in graph.get(node, []):
            if neighbor not in visited:
                new_g = g + weight
                new_f = new_g + heuristic.get(neighbor, 0)
                heapq.heappush(frontier, (new_f, new_g, path + [neighbor]))
    return None, float('inf')


if __name__ == "__main__":
    start, goal = 'Kolkata', 'Ranchi'
```

```
    print("BFS  path (fewest hops):        ", bfs(start, goal))
    print("DFS  path (first found, deep):  ", dfs(start, goal))

    ucs_path, ucs_cost = uniform_cost_search(start, goal)
    print(f"UCS  path (lowest cost):         {ucs_path}, "
          f"total distance = {ucs_cost} km")

    astar_path, astar_cost = a_star_search(start, goal, heuristic_to_ranchi)
    print(f"A*   path (guided by heuristic): {astar_path}, "
          f"total distance = {astar_cost} km")
```

Expected output:

```
BFS  path (fewest hops):        ['Kolkata', 'Asansol', 'Dhanbad', 'Ranchi']
DFS  path (first found, deep):   ['Kolkata', 'Durgapur', 'Asansol',
 ↪  'Dhanbad', 'Ranchi']
UCS  path (lowest cost):         ['Kolkata', 'Asansol', 'Dhanbad',
 ↪  'Ranchi'], total distance = 420 km
A*   path (guided by heuristic): ['Kolkata', 'Asansol', 'Dhanbad',
 ↪  'Ranchi'], total distance = 420 km
```

What to notice: BFS and DFS only care about the *number of steps* or *finding any path*, so they can return a longer route by distance. UCS and A* both find the truly shortest route (420 km) — but A* gets there by exploring **fewer nodes**, because the heuristic guides it toward the goal instead of blindly exploring everything, exactly like a real GPS narrowing down routes quickly instead of checking every road in the country.

Try it yourself: change the graph to your own city/college campus map, or add more cities, and watch how each algorithm behaves differently.

Chapter 3

Module 2: Logic and Deduction

3.1 2.1 Why Logic Matters in AI

Before machine learning became dominant, AI systems reasoned using formal **logic** — precise rules for representing facts and deriving new facts from them. Logic is still the backbone of rule engines, theorem provers, verification tools, and the "reasoning core" of expert systems (e.g., legal-reasoning software, tax-calculation engines, and modern-day fraud rule systems).

Real-life example: A loan-approval rule engine at a bank might encode: *"IF credit_score > 750 AND income > 500000 THEN approve_loan."* This is propositional logic in action, still used today alongside ML models.

3.2 2.2 Propositional Logic

A **proposition** is a statement that is either **True** or **False**, never both. *Examples:* "It is raining" (P), "The exam is on Monday" (Q).

3.2.1 2.2.1 Logical Connectives

Connective	Symbol	Meaning	Real-life phrasing
Negation	¬P	NOT P	"It is not raining"
Conjunction	P ∧ Q	P AND Q	"It is raining AND it is cold"
Disjunction	P ∨ Q	P OR Q	"I will study OR I will sleep"
Exclusive OR	P ⊕ Q	Either, not both	"You get tea XOR coffee"
Implication	P → Q	IF P THEN Q	"IF it rains THEN the ground is wet"

Connective	Symbol	Meaning	Real-life phrasing
Biconditional	P ↔ Q	P IF AND ONLY IF Q	“The alarm rings IFF the door opens”

3.2.2 2.2.2 Truth Tables

Negation:

P	¬P
T	F
F	T

Conjunction (AND):

P	Q	P ∧ Q
T	T	T
T	F	F
F	T	F
F	F	F

Disjunction (OR):

P	Q	P ∨ Q
T	T	T
T	F	T
F	T	T
F	F	F

Implication (IF…THEN):

P	Q	P → Q
T	T	T
T	F	F
F	T	T
F	F	T

Common confusion: `P → Q` is only False when P is True and Q is False. If P is False, the implication is *vacuously* True — “if pigs fly, I’ll eat my hat” is technically true today because pigs don’t fly!

Biconditional (IFF):

P	Q	P ↔ Q
T	T	T
T	F	F
F	T	F
F	F	T

3.2.3 2.2.3 Satisfiability, Validity, and Contradiction

- **Satisfiable:** there is *at least one* assignment of T/F that makes the statement True. *Example: $P \wedge Q$ is satisfiable (true when both P, Q are true).*
- **Valid / Tautology:** True under **every** assignment. *Example: $P \vee \neg P$ (the "law of excluded middle") is always true.*
- **Contradiction:** False under **every** assignment. *Example: $P \wedge \neg P$ can never be true.*

Real-life use: SAT solvers (satisfiability solvers) built on these ideas are used today in chip design verification, software bug detection, and scheduling problems (e.g., timetable generation for a university — "can we satisfy every constraint simultaneously?").

3.3 2.3 Predicate Logic (First-Order Logic)

Propositional logic cannot express statements about *objects* and their *properties* — for that we need **predicate logic**.

- **Predicate:** a function that returns True/False about an object. *Example: `IsStudent(x)`, `Likes(x, y)`.*
- **Quantifiers:**
  - **Universal (∀):** "for all." *Example: $\forall x$ IsStudent(x) → AttendsClass(x) means "All students attend class."*
  - **Existential (∃):** "there exists." *Example: $\exists x$ IsStudent(x) ∧ Likes(x, AI) means "Some student likes AI."*

Real-life example: A hospital database rule "∀patient (HasFever(patient) ∧ HasCough(patient)) → Recommend(patient, COVID_Test)" is predicate logic driving a clinical decision-support system.

3.4 2.4 Inference — Deriving New Knowledge

Given known facts and rules, an inference engine derives new facts.

- **Modus Ponens:** From `P → Q` and `P`, conclude `Q`. *Example: "If it rains, the ground is wet" + "It is raining" ⟹ "The ground is wet."*
- **Modus Tollens:** From `P → Q` and `¬Q`, conclude `¬P`. *Example: "If it rains, the ground is wet" + "The ground is NOT wet" ⟹ "It did NOT rain."*

- **Resolution:** A single rule of inference used by automated theorem provers (and Prolog) to derive a contradiction, proving a statement by *refutation*.

Real-life example: Expert systems like MYCIN (medical diagnosis) chained together dozens of such IF-THEN rules using inference engines to reach a diagnosis, the same core idea powering today's business rule engines (Drools, insurance-claim approval systems).

3.5 2.5 PRACTICAL: Propositional Logic in Python

3.5.1 2.5.1 Truth Table Generator

```python
# truth_table.py
import itertools

def evaluate(expr_func, variables):
    """Print a full truth table for a boolean expression."""
    header = list(variables) + ["Result"]
    print(" | ".join(header))
    print("-" * (len(header) * 8))
    for values in itertools.product([True, False], repeat=len(variables)):
        row = dict(zip(variables, values))
        result = expr_func(**row)
        printable = [str(row[v]) for v in variables] + [str(result)]
        print(" | ".join(printable))

# Example 1: Implication  P -> Q   (equivalent to: not P or Q)
print("Truth table for P -> Q:")
evaluate(lambda P, Q: (not P) or Q, ["P", "Q"])

print("\nTruth table for (P AND Q) OR (NOT P):")
evaluate(lambda P, Q: (P and Q) or (not P), ["P", "Q"])
```

Expected output (first table):

```
Truth table for P -> Q:
P | Q | Result
------------------------
True | True | True
True | False | False
False | True | True
False | False | True
```

3.5.2 2.5.2 A Tiny Forward-Chaining Inference Engine

This mimics how a rule-based expert system reaches a conclusion — the same mechanism (in principle) used by MYCIN-style medical diagnosis systems and modern business-rule engines.

```python
# inference_engine.py

# Knowledge base: rules are (list_of_conditions, conclusion)
rules = [
    (["has_fever", "has_cough"], "possible_flu"),
    (["possible_flu", "recent_travel"], "recommend_test"),
    (["has_rash", "has_fever"], "possible_measles"),
]

def forward_chain(facts, rules):
    """Repeatedly apply rules until no new facts can be derived."""
    facts = set(facts)
    added = True
    while added:
        added = False
        for conditions, conclusion in rules:
            if all(c in facts for c in conditions) and conclusion not in
            ↪  facts:
                facts.add(conclusion)
                print(f"Derived new fact: '{conclusion}'  "
                      f"(from rule: {conditions} -> {conclusion})")
                added = True
    return facts

if __name__ == "__main__":
    known_facts = ["has_fever", "has_cough", "recent_travel"]
    print(f"Starting facts: {known_facts}\n")
    final_facts = forward_chain(known_facts, rules)
    print(f"\nAll facts after inference: {sorted(final_facts)}")
```

Expected output:

```
Starting facts: ['has_fever', 'has_cough', 'recent_travel']

Derived new fact: 'possible_flu'  (from rule: ['has_fever', 'has_cough'] ->
 ↪  possible_flu)
Derived new fact: 'recommend_test'  (from rule: ['possible_flu',
 ↪  'recent_travel'] -> recommend_test)

All facts after inference: ['has_cough', 'has_fever', 'possible_flu',
 ↪  'recent_travel', 'recommend_test']
```

What's happening: this is exactly the logic that runs inside symptom-checker apps

(like WebMD’s or Ada Health’s triage bots) before any machine-learning model gets involved — a chain of IF-THEN rules mechanically producing new conclusions from known facts.

Chapter 4

Module 3: Introduction to Machine Learning

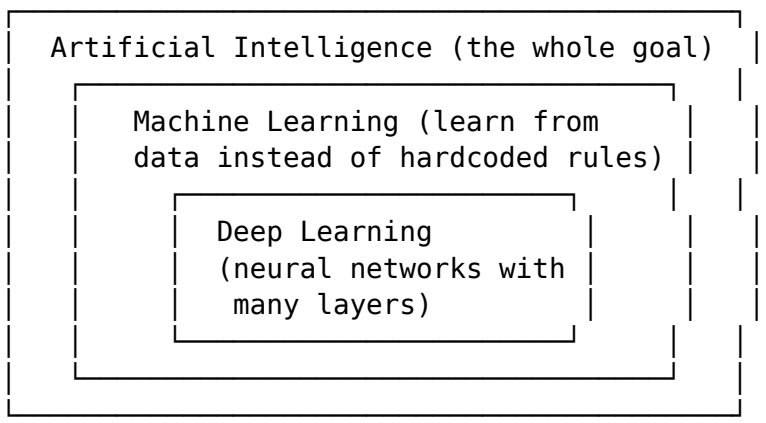
4.1 3.1 What is Machine Learning?

Machine Learning (ML) is a subfield of AI where, instead of a programmer writing explicit rules, the machine **learns patterns from data** and improves its performance through experience.

> Classic definition (Tom Mitchell): *A program learns from experience E with respect to task T and performance measure P, if its performance at T, measured by P, improves with E.*

Real-life anchor: Instead of hand-writing 10,000 rules to detect spam email ("if subject contains 'FREE MONEY' then spam"), you show the algorithm thousands of labeled emails (spam / not-spam) and it *learns* the patterns itself — including patterns a human might never think to write as a rule.

4.1.1 AI vs. ML vs. Deep Learning



4.2 3.2 Types of Machine Learning

4.2.1 3.2.1 Supervised Learning

The model learns from **labeled data** — each training example has an input *and* the correct output.

Real-life example: Building an image classifier that tells apart cats vs. dogs. You feed it thousands of images *already labeled* "cat" or "dog." The model learns the mapping from pixels → label, then predicts labels on brand-new, unseen images.

Two main tasks: - **Classification:** predict a category (spam/not-spam, cat/dog, disease/no-disease). - **Regression:** predict a continuous number (house price, temperature, stock value).

4.2.2 3.2.2 Unsupervised Learning

The model works with **unlabeled data** — it must find hidden structure on its own.

Real-life example: An e-commerce company has millions of customers but no predefined "customer type" labels. Unsupervised learning (clustering) groups customers into segments — "bargain hunters," "brand loyalists," "occasional buyers" — purely based on shopping patterns, letting the marketing team target each group differently.

4.2.3 3.2.3 Reinforcement Learning (RL)

An **agent** learns by interacting with an **environment**, receiving **rewards** for good actions and **penalties** for bad ones, with no labeled dataset at all.

- **Positive reinforcement:** rewards a desired action, encouraging repetition. *Example: giving a dog a treat for sitting.*
- **Negative reinforcement:** removes an unpleasant stimulus when the desired behavior occurs. *Example: turning off a loud seatbelt alarm once you buckle up.*

Real-life examples: - **Game playing:** AlphaGo learned to play Go by playing millions of games against itself. - **Robotics:** robots learn to walk or grasp objects through trial and error. - **Self-driving cars:** learning driving policies through simulated trial-and-error. - **Recommendation systems, algorithmic trading, energy-grid optimization, adaptive personal assistants** all use RL in production today.

Trade-off: RL can solve problems no other technique can, but it is computationally expensive, needs huge amounts of interaction data, and is overkill for simple problems that supervised learning would solve more cheaply.

4.3 3.3 The Machine Learning Workflow

1. **Collect data** — e.g., historical house sale prices.
2. **Clean & prepare data** — handle missing values, remove outliers, encode categories.
3. **Split data** — training set (learn from it) and test set (evaluate on unseen data).

4. **Choose a model** — linear regression, decision tree, neural network, etc.
5. **Train the model** — the algorithm adjusts internal parameters to fit the training data.
6. **Evaluate** — measure accuracy/error on the test set.
7. **Tune & deploy** — adjust hyperparameters, then put the model into production.

4.4 3.4 Regression Basics (Foundational Math for ML)

4.4.1 3.4.1 Polynomial Interpolation

Given a set of data points, find a polynomial curve that passes *exactly* through every point. Useful for small, noise-free datasets — e.g., fitting a curve through exact sensor calibration readings.

4.4.2 3.4.2 Least Squares Fitting

When data is noisy (real-world data almost always is), we don't want a curve through every point exactly — we want the curve that **minimizes the total squared error** between predictions and actual values.

$$\text{minimize} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Real-life example: Predicting a student's exam score from hours studied. Not every student who studies 5 hours gets exactly the same score (noise from other factors), so we fit the *best-fitting line*, not a line through every single point.

4.5 3.5 Probability Foundations: Bayes' Theorem

Bayes' Theorem lets us update our belief about something as new evidence arrives:

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

Disease-test example (a classic, high-value real-world case): Suppose a disease affects 1% of a population. A test is 99% accurate (both for sick and healthy people). If you test positive, what's the actual chance you have the disease?

Intuition says "99%," but Bayes' Theorem reveals something surprising: because the disease is rare, most positive results are actually **false positives** from the healthy 99% of the population. Working through the numbers (with 10,000 people: 100 sick, 9,900 healthy): - True positives: 100 × 0.99 = 99 - False positives: 9,900 × 0.01 = 99 - P(disease | positive test) = 99 / (99 + 99) = **50%**, not 99%!

This is exactly why doctors don't diagnose from a single test alone, and why understanding Bayesian reasoning matters enormously in medicine, spam filtering, and legal reasoning (avoiding the "prosecutor's fallacy").

4.5.1 Naive Bayes Classifier

A practical ML classifier built directly on Bayes' Theorem, "naively" assuming all features are independent given the class. Despite this simplifying (and often technically wrong) assumption, it works remarkably well in practice.

Real-life example: Spam filters have used Naive Bayes for decades — it estimates P(spam | words in email) using the frequency of words like "free," "winner," "click here" in known spam vs. legitimate emails.

4.6 3.6 PRACTICAL: Regression and Naive Bayes in Python

4.6.1 3.6.1 Least Squares Linear Regression From Scratch

```python
# linear_regression_scratch.py
import numpy as np

# Real-life style dataset: hours studied vs. exam score (out of 100)
hours_studied = np.array([1, 2, 3, 4, 5, 6, 7, 8])
exam_score    = np.array([35, 40, 50, 55, 65, 70, 78, 85])

def least_squares_fit(x, y):
    """Compute slope (m) and intercept (c) for y = m*x + c using the
    closed-form least-squares formulas."""
    n = len(x)
    x_mean, y_mean = x.mean(), y.mean()
    m = np.sum((x - x_mean) * (y - y_mean)) / np.sum((x - x_mean) ** 2)
    c = y_mean - m * x_mean
    return m, c

m, c = least_squares_fit(hours_studied, exam_score)
print(f"Learned model: score = {m:.2f} * hours + {c:.2f}")

# Predict the score for a new student who studies 4.5 hours
new_hours = 4.5
predicted = m * new_hours + c
print(f"Predicted score for {new_hours} hours studied: {predicted:.2f}")
```

Expected output:

```
Learned model: score = 7.26 * hours + 27.07
Predicted score for 4.5 hours studied: 59.75
```

4.6.2 3.6.2 Naive Bayes Spam Classifier (using scikit-learn)

```python
# naive_bayes_spam.py
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# A tiny labeled dataset: 1 = spam, 0 = not spam (ham)
emails = [
    "win a free lottery prize now",
    "meeting scheduled for tomorrow at 10am",
    "claim your free cash prize today",
    "please review the attached project report",
    "you have won a free vacation click here",
    "let's catch up over lunch next week",
]
labels = [1, 0, 1, 0, 1, 0]

# Convert text into word-count vectors the model can learn from
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(emails)

model = MultinomialNB()
model.fit(X, labels)

# Test on new, unseen emails
test_emails = [
    "free prize waiting for you, click now",
    "project deadline moved to next monday",
]
X_test = vectorizer.transform(test_emails)
predictions = model.predict(X_test)

for email, pred in zip(test_emails, predictions):
    label = "SPAM" if pred == 1 else "NOT SPAM"
    print(f"'{email}'  ->  {label}")
```

Expected output:

```
'free prize waiting for you, click now'  ->  SPAM
'project deadline moved to next monday'  ->  NOT SPAM
```

Why this matters in real life: This tiny example is the same core idea (word frequency + Bayes' theorem) that powered early Gmail/Outlook spam filters before deep learning became affordable. It's cheap to train, fast to run, and still used today as a baseline model in text classification pipelines.

Chapter 5

Module 4: Supervised Learning and Unsupervised Learning

5.1 4.1 Decision Trees

A **Decision Tree** predicts an outcome by asking a series of yes/no (or multi-way) questions, forming a tree of decisions.

Real-life example: A bank loan approval flowchart — "Is income > 5 LPA? → Yes → Is credit score > 700? → Yes → Approve." Decision trees literally look like flowcharts humans already use, which is why they are so easy to explain to non-technical stakeholders (a huge advantage over "black box" models).

5.1.1 4.1.1 Entropy and Information Gain (the ID3 Algorithm)

Entropy measures the "impurity" or disorder in a set of examples:

$$H(S) = -\sum_{i} p_i \log_2 p_i$$

- Entropy = 0 when all examples belong to one class (perfectly "pure").
- Entropy = 1 (max, for 2 classes) when classes are perfectly mixed 50/50.

Information Gain measures how much entropy is reduced by splitting on a particular attribute:

$$IG(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

5.1.2 ID3 Algorithm (builds the tree)

```
1. Create a root node for the tree.
2. If all examples are positive, return a leaf node "positive".
3. If all examples are negative, return a leaf node "negative".
```

```
4. Calculate the entropy of the current state H(S).
5. For each attribute, calculate entropy after splitting, H(S, x).
6. Select the attribute with the maximum Information Gain, IG(S, x).
7. Remove that attribute from further consideration below this branch.
8. Repeat until all attributes are used or all leaves are pure.
```

Real-life example: Deciding "should I play tennis today?" based on Outlook (sunny/overcast/rain), Temperature, Humidity, and Wind — the textbook example that mirrors real weather-based scheduling systems (e.g., outdoor-event planning apps).

5.2 4.2 Linear Regression (Supervised — Regression Task)

5.2.1 4.2.1 Simple Linear Regression

One input feature predicts one output: `y = mx + c`. **Real-life example:** predicting monthly electricity bill (y) from number of AC hours used per day (x).

5.2.2 4.2.2 Multiple Linear Regression

Several input features predict one output: `y = b0 + b1x1 + b2x2 + ... + bnxn`. **Real-life example:** predicting a house's price from square footage, number of bedrooms, location score, and age of the property — exactly how real estate pricing tools like Zillow's "Zestimate" work at their core (with much more sophisticated models layered on top).

5.2.3 4.2.3 Positive vs. Negative Linear Relationships

- **Positive relationship:** as x increases, y increases (e.g., more study hours → higher exam score).
- **Negative relationship:** as x increases, y decreases (e.g., more speed → less travel time for a fixed distance).

5.3 4.3 Unsupervised Learning: Clustering

Clustering groups similar data points together **without any labels**.

5.3.1 4.3.1 K-Means Clustering

The most widely used clustering algorithm.

Algorithm:

```
1. Choose K (the number of clusters).
2. Randomly initialize K cluster centers ("centroids").
3. Repeat until convergence:
   a. Assign each data point to its nearest centroid.
   b. Recompute each centroid as the mean of points assigned to it.
```

Real-life example: A retail chain wants to segment its 100,000 customers into groups for targeted marketing. K-Means groups customers by purchase frequency and average spend into clusters like "high-value frequent shoppers," "occasional big spenders," and "rare small purchasers" — with zero manual labeling needed.

5.3.2 4.3.2 Hierarchical Clustering

Builds a tree (dendrogram) of nested clusters, either by: - **Agglomerative (bottom-up):** start with every point as its own cluster, repeatedly merge the closest pair. - **Divisive (top-down):** start with one giant cluster, repeatedly split it.

Real-life example: Biologists use hierarchical clustering to build phylogenetic trees showing how species are related by genetic similarity — no predefined number of clusters is needed, unlike K-Means.

5.3.3 Supervised vs. Unsupervised — Side by Side

Aspect	Supervised	Unsupervised
Data	Labeled	Unlabeled
Goal	Predict a known output	Discover hidden structure
Example algorithms	Linear Regression, Decision Trees, SVM	K-Means, Hierarchical Clustering
Real-life use	Credit scoring, disease diagnosis	Customer segmentation, anomaly detection

---

5.4 4.4 PRACTICAL: Decision Tree and K-Means in Python

5.4.1 4.4.1 Decision Tree Classifier (scikit-learn) — "Should I Play Tennis?"

```
# decision_tree_tennis.py
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_text

# Classic "Play Tennis" dataset
data = {
    "Outlook": [
        "Sunny", "Sunny", "Overcast", "Rain", "Rain",
        "Rain", "Overcast", "Sunny", "Sunny", "Rain",
    ],
    "Temperature": [
        "Hot", "Hot", "Hot", "Mild", "Cool",
        "Cool", "Cool", "Mild", "Cool", "Mild",
```

```
    ],
    "Humidity": [
        "High", "High", "High", "High", "Normal",
        "Normal", "Normal", "High", "Normal", "Normal",
    ],
    "Wind": [
        "Weak", "Strong", "Weak", "Weak", "Weak",
        "Strong", "Strong", "Weak", "Weak", "Weak",
    ],
    "PlayTennis": [
        "No", "No", "Yes", "Yes", "Yes",
        "No", "Yes", "No", "Yes", "Yes",
    ],
}
df = pd.DataFrame(data)

# Encode text categories into numbers (decision trees in sklearn need
 ↪  numeric input)
encoders = {}
df_encoded = df.copy()
for col in df.columns:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df[col])
    encoders[col] = le

X = df_encoded.drop("PlayTennis", axis=1)
y = df_encoded["PlayTennis"]

model = DecisionTreeClassifier(criterion="entropy", random_state=0)
model.fit(X, y)

print("Learned decision tree rules:\n")
print(export_text(model, feature_names=list(X.columns)))

# Predict for a brand-new day: Sunny, Mild, Normal humidity, Weak wind
new_day = pd.DataFrame({
    "Outlook": [encoders["Outlook"].transform(["Sunny"])[0]],
    "Temperature": [encoders["Temperature"].transform(["Mild"])[0]],
    "Humidity": [encoders["Humidity"].transform(["Normal"])[0]],
    "Wind": [encoders["Wind"].transform(["Weak"])[0]],
})
prediction = model.predict(new_day)
print("\nPrediction for (Sunny, Mild, Normal, Weak):",
      encoders["PlayTennis"].inverse_transform(prediction)[0])
```

Expected output (exact thresholds depend on sklearn's internal alphabetical encoding of category names — class `0` = "No", class `1` = "Yes"):

```
Learned decision tree rules:

|--- Outlook <= 1.50
|   |--- Wind <= 0.50
|   |   |--- Outlook <= 0.50
|   |   |   |--- class: 1
|   |   |--- Outlook >  0.50
|   |   |   |--- class: 0
|   |--- Wind >  0.50
|   |   |--- class: 1
|--- Outlook >  1.50
|   |--- Temperature <= 0.50
|   |   |--- class: 1
|   |--- Temperature >  0.50
|   |   |--- class: 0

Prediction for (Sunny, Mild, Normal, Weak): No
```

Note on such a tiny dataset: with only 10 training rows the tree is prone to overfitting to quirks in this exact sample — in a real deployment you would train on thousands of days of historical weather + play/no-play records, and validate on a separate held-out test set before trusting its predictions.

5.4.2 4.4.2 K-Means Customer Segmentation

```python
# kmeans_customers.py
import numpy as np
from sklearn.cluster import KMeans

# Real-life style data: [annual_spend_in_thousands, visits_per_month]
customers = np.array([
    [80, 2],   # big spender, rare visitor
    [90, 1],
    [85, 2],
    [20, 15],  # frequent small-basket shopper
    [22, 18],
    [18, 16],
    [50, 8],   # medium spender, regular visitor
    [55, 7],
    [48, 9],
])

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(customers)

print("Cluster assignment for each customer:", kmeans.labels_)
print("Cluster centers (spend, visits):\n", kmeans.cluster_centers_)
```

```python
# Classify a brand-new customer
new_customer = np.array([[85, 3]])
predicted_cluster = kmeans.predict(new_customer)
print(f"\nNew customer {new_customer.tolist()} "
      f"belongs to cluster: {predicted_cluster[0]}")
```

Expected output (cluster *numbers* are arbitrary labels assigned by the algorithm, but the three groupings are consistent):

```
Cluster assignment for each customer: [1 1 1 0 0 0 2 2 2]
Cluster centers (spend, visits):
 [[20.         16.33333333]
 [85.          1.66666667]
 [51.          8.        ]]

New customer [[85, 3]] belongs to cluster: 1
```

Why this matters: the marketing team can now message cluster 1 (“high spend, low frequency”) with “come back and get 10% off,” cluster 0 (“frequent, small basket”) with bulk-discount offers, and cluster 2 (“steady medium spenders”) differently again — all discovered automatically, with no manual labeling of a single customer.

Chapter 6

Module 5: Artificial Neural Networks and Genetic Algorithms

6.1 5.1 Biological Inspiration

An **Artificial Neural Network (ANN)** is loosely inspired by how neurons in the brain connect and fire. A biological neuron receives signals through dendrites, processes them in the cell body, and fires an output signal through the axon if the combined input crosses a threshold. An artificial "neuron" (perceptron) mimics this: it takes weighted inputs, sums them, and passes the result through an **activation function**.

6.2 5.2 The Perceptron — The Simplest Neural Network

$$\text{output} = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

- `x_i` = inputs
- `w_i` = weights (learned during training — how important each input is)
- `b` = bias (shifts the decision boundary)
- `f` = activation function (decides whether the neuron "fires")

Real-life analogy: deciding "should I go to the party tonight?" You weigh inputs — "Is it a close friend's party?" (weight: high), "Do I have an exam tomorrow?" (weight: very high, negative), "Is it far away?" (weight: medium, negative) — sum them up, and if the total crosses your personal threshold, you go. A perceptron formalizes exactly this weighted decision process.

6.2.1 Common Activation Functions

Function	Formula	Real-life use
Sigmoid	1 / (1 + e^-x)	Squashes output to (0,1) — used for probability-like outputs, e.g., “probability this email is spam”
ReLU	max(0, x)	Most common in deep networks today — fast, avoids “vanishing gradient”
Tanh	(e^x − e$^{-x)/(e}$x + e^-x)	Squashes to (-1,1), used in some recurrent networks
Softmax	normalizes outputs into a probability distribution	Used in the final layer for multi-class classification, e.g., “which of these 10 digits is this?”

6.3 5.3 Multi-Layer Neural Networks and Backpropagation

A single perceptron can only learn linearly separable patterns. Stacking layers of neurons (an **input layer**, one or more **hidden layers**, and an **output layer**) lets a network learn far more complex, non-linear patterns — this is the basis of **deep learning**.

6.3.1 How training works (Backpropagation)

1. **Forward pass:** input flows through the network to produce a prediction.
2. **Compute loss:** compare prediction to the true label using a loss function (e.g., mean squared error, cross-entropy).
3. **Backward pass (backpropagation):** using calculus (the chain rule), compute how much each weight contributed to the error.
4. **Gradient Descent:** nudge every weight slightly in the direction that reduces the error.
5. Repeat over many examples and many passes (“epochs”) until the error is acceptably small.

Real-life example: Training a handwriting-digit recognizer (like the classic MNIST task, used in real postal code readers and bank cheque processing) — the network sees thousands of labeled digit images, and backpropagation gradually adjusts millions of weights until it reliably tells a “7” from a “1.”

6.4 5.4 Other Key Supervised Algorithms

6.4.1 5.4.1 K-Nearest Neighbors (KNN)

Classifies a new point by looking at the `K` closest labeled points and taking a majority vote.

Real-life example: Netflix-style "users similar to you also liked…" recommendations — find the K most similar users (by viewing history) and recommend what they liked.

6.4.2 5.4.2 Support Vector Machines (SVM)

Finds the **best separating boundary (hyperplane)** between two classes — specifically, the one that maximizes the margin (distance) to the nearest points of each class ("support vectors"). For data that isn't linearly separable, SVM uses the **kernel trick** to project data into a higher dimension where it becomes separable.

Real-life example: SVMs were historically the go-to method for text classification (e.g., sentiment analysis: "is this movie review positive or negative?") and bioinformatics (classifying gene expression data) before deep learning became dominant.

6.5 5.5 Genetic Algorithms (GA)

A **Genetic Algorithm** is an optimization technique inspired by **natural selection** — "survival of the fittest" applied to solving problems.

6.5.1 Why use GAs?

1. Evolution is a proven, robust method for adaptation in complex, changing environments.
2. GAs can search huge, complex solution spaces where the pieces interact in complicated ways (hard for traditional calculus-based optimization).
3. GAs are easily parallelized, taking advantage of cheap, distributed computing.

6.5.2 The GA Algorithm

```
1. Initialize a random population of candidate solutions ("chromosomes").
2. Evaluate each candidate's "fitness" (how good a solution it is).
3. Select the fittest candidates to be "parents" (e.g., roulette-wheel or
 ↪  tournament selection).
4. Crossover: combine pairs of parents to produce "offspring" (mix parts of
 ↪  each solution).
5. Mutation: randomly tweak a small part of some offspring to maintain
 ↪  diversity.
6. Replace the old population with the new generation.
7. Repeat steps 2–6 for many generations until a good-enough solution is
 ↪  found.
```

Real-life examples: - **Engineering design:** NASA used a genetic algorithm to design an unconventional but highly efficient antenna shape for a spacecraft — a shape no human engineer would have intuitively designed. - **Scheduling:** airlines and factories use GAs to optimize staff rosters and production schedules where there are too many constraints and combinations for brute-force search. - **Game AI & procedural content:** evolving game-playing strategies or level designs. - **Route optimization:** solving variants of the Traveling Salesman Problem for delivery-truck routing (e.g., logistics companies optimizing daily delivery routes).

6.6 5.6 PRACTICAL: Neural Network and Genetic Algorithm in Python

6.6.1 5.6.1 A Neural Network From Scratch (NumPy Only) — Learning the XOR Function

XOR is the classic example that a *single* perceptron cannot learn (it isn't linearly separable), which is exactly why multi-layer networks were needed historically.

```
# neural_network_xor.py
import numpy as np

np.random.seed(42)

# XOR truth table as training data
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Network architecture: 2 inputs -> 4 hidden neurons -> 1 output
input_size, hidden_size, output_size = 2, 4, 1

W1 = np.random.uniform(-1, 1, (input_size, hidden_size))
b1 = np.zeros((1, hidden_size))
W2 = np.random.uniform(-1, 1, (hidden_size, output_size))
b2 = np.zeros((1, output_size))

learning_rate = 0.5
epochs = 10000

for epoch in range(epochs):
    # ---- Forward pass ----
```

```
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + b2
    predicted = sigmoid(final_input)

    # ---- Compute error ----
    error = y - predicted

    # ---- Backward pass (backpropagation) ----
    d_predicted = error * sigmoid_derivative(predicted)
    error_hidden = d_predicted.dot(W2.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    # ---- Update weights (gradient descent step) ----
    W2 += hidden_output.T.dot(d_predicted) * learning_rate
    b2 += np.sum(d_predicted, axis=0, keepdims=True) * learning_rate
    W1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

    if epoch % 2000 == 0:
        loss = np.mean(np.square(error))
        print(f"Epoch {epoch:5d}  |  Loss: {loss:.5f}")

print("\nFinal predictions after training:")
for inputs, pred in zip(X, predicted):
    print(f"Input: {inputs}  ->  Predicted: {pred[0]:.3f}  "
          f"(rounded: {round(pred[0])})")
```

Expected output (values will vary slightly, but the pattern will match):

```
Epoch     0  |  Loss: 0.26292
Epoch  2000  |  Loss: 0.00179
Epoch  4000  |  Loss: 0.00070
Epoch  6000  |  Loss: 0.00043
Epoch  8000  |  Loss: 0.00031

Final predictions after training:
Input: [0 0]  ->  Predicted: 0.009  (rounded: 0)
Input: [0 1]  ->  Predicted: 0.984  (rounded: 1)
Input: [1 0]  ->  Predicted: 0.985  (rounded: 1)
Input: [1 1]  ->  Predicted: 0.019  (rounded: 0)
```

What's happening: the network starts with random weights (guessing badly, high loss). Over thousands of epochs, backpropagation + gradient descent nudge the weights until the hidden layer learns an internal representation that makes XOR linearly separable — something no single-layer perceptron could ever do. This exact principle, scaled up to millions of neurons and billions of parameters, underlies today's large image- and language-models.

6.6.2 5.6.2 A Genetic Algorithm — Optimizing a Delivery Route (Mini Traveling Salesman)

```python
# genetic_algorithm_route.py
import random

random.seed(1)

# Coordinates of delivery stops (like a small delivery route for a courier)
cities = {
    "Depot": (0, 0), "A": (2, 4), "B": (5, 2), "C": (8, 6),
    "D": (3, 8), "E": (7, 1),
}
city_names = list(cities.keys())

def route_distance(route):
    """Total distance of visiting cities in this order and returning to
    ↪ start."""
    total = 0
    for i in range(len(route)):
        x1, y1 = cities[route[i]]
        x2, y2 = cities[route[(i + 1) % len(route)]]
        total += ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5
    return total

def create_individual():
    """A random ordering of cities (excluding fixed starting depot)."""
    others = city_names[1:]
    random.shuffle(others)
    return ["Depot"] + others

def fitness(route):
    """Shorter distance = higher fitness."""
    return 1 / route_distance(route)

def crossover(parent1, parent2):
    """Ordered crossover: keep a slice from parent1, fill the
    rest from parent2's order."""
    size = len(parent1)
    start, end = sorted(random.sample(range(1, size), 2))
    child = [None] * size
    child[0] = "Depot"
    child[start:end] = parent1[start:end]
    fill_values = [c for c in parent2 if c not in child]
    fill_positions = [i for i in range(1, size) if child[i] is None]
    for pos, val in zip(fill_positions, fill_values):
        child[pos] = val
```

```
    return child

def mutate(route, rate=0.2):
    """Randomly swap two cities (excluding the fixed depot at position
    ↪ 0)."""
    if random.random() < rate:
        i, j = random.sample(range(1, len(route)), 2)
        route[i], route[j] = route[j], route[i]
    return route

def genetic_algorithm(pop_size=50, generations=200):
    population = [create_individual() for _ in range(pop_size)]

    for gen in range(generations):
        population.sort(key=route_distance)
        if gen % 40 == 0:
            best = population[0]
            print(f"Generation {gen:3d}  |  "
                  f"Best distance so far: {route_distance(best):.2f}  |  "
                  f"Route: {best}")

        # Elitism: keep the best 10% automatically
        next_gen = population[: pop_size // 10]

        # Breed the rest of the new generation
        while len(next_gen) < pop_size:
            parent1, parent2 = random.sample(population[: pop_size // 2], 2)
            child = crossover(parent1, parent2)
            child = mutate(child)
            next_gen.append(child)

        population = next_gen

    population.sort(key=route_distance)
    return population[0], route_distance(population[0])

if __name__ == "__main__":
    best_route, best_distance = genetic_algorithm()
    print(f"\nBest route found: {best_route}")
    print(f"Total distance: {best_distance:.2f}")
```

Expected output (exact numbers vary with randomness, but distance should steadily decrease):

```
Generation   0  |  Best distance so far: 28.29  |  Route: ['Depot', 'A',
↪  'D', 'C', 'B', 'E']
Generation  40  |  Best distance so far: 26.70  |  Route: ['Depot', 'A',
↪  'D', 'C', 'E', 'B']
```

```
Generation  80  |  Best distance so far: 26.70  |  Route: ['Depot', 'A',
 ↪  'D', 'C', 'E', 'B']
Generation 120  |  Best distance so far: 26.70  |  Route: ['Depot', 'A',
 ↪  'D', 'C', 'E', 'B']
Generation 160  |  Best distance so far: 26.70  |  Route: ['Depot', 'A',
 ↪  'D', 'C', 'E', 'B']

Best route found: ['Depot', 'A', 'D', 'C', 'E', 'B']
Total distance: 26.70
```

Why this matters in real life: this toy example scales directly to real logistics — companies like FedEx, Amazon, and food-delivery apps use genetic algorithms (and related metaheuristics) to optimize thousands of delivery routes daily, because an exact brute-force solution for even 20 stops would take longer than the age of the universe to compute, while a GA finds a *very good* route in seconds.

6.7 5.7 Module 5 Summary Table

Concept	One-line takeaway	Real-world system that uses it
Perceptron	Weighted sum + threshold decision	Simple binary classifiers
Multi-layer ANN + Backprop	Learns complex, non-linear patterns	Image recognition, speech-to-text
KNN	“You are like your neighbors”	Recommendation systems
SVM	Best boundary with max margin	Text/sentiment classification
Genetic Algorithm	Evolve solutions via selection, crossover, mutation	Route optimization, engineering design, scheduling

Chapter 7

Quick Revision: The Whole Course in One Table

Module	Core Topics	Key Algorithms	Real-World Examples
1. AI Concepts & Search	History, AI winters, agents, PEAS, knowledge representation	BFS, DFS, UCS, Greedy, A*	GPS routing, game NPC pathfinding
2. Logic & Deduction	Propositional & predicate logic, truth tables, inference	Modus Ponens, Resolution, Forward Chaining	Rule engines, expert systems, SAT solvers
3. Intro to ML	Supervised/unsupervised/reinforcement learning, regression basics, Bayes' theorem	Least Squares, Naive Bayes	Spam filters, medical test interpretation
4. Supervised & Unsupervised	Decision trees, entropy, linear regression, clustering	ID3, K-Means, Hierarchical Clustering	Loan approval, customer segmentation
5. ANN & Genetic Algorithm	Perceptron, backpropagation, KNN, SVM, evolutionary search	Multi-layer ANN, Genetic Algorithm	Image recognition, delivery route optimization

Chapter 8

Glossary of Key Terms

Agent — Anything that perceives its environment and acts upon it.

Backpropagation — The algorithm that computes how to adjust a neural network's weights by propagating the output error backward through the layers.

Bias (ML) — A model's tendency to consistently miss the true pattern (underfitting); also, in a neuron, the constant added to the weighted sum.

Classification — Predicting a discrete category label (e.g., spam / not spam).

Clustering — Grouping similar unlabeled data points together.

Entropy — A measure of impurity/disorder in a set of examples; 0 = perfectly pure.

Epoch — One complete pass of the training data through a learning algorithm.

Feature — An individual measurable input variable used by a model (e.g., "square footage" for house price prediction).

Heuristic — A rule-of-thumb estimate used to guide search toward a goal faster.

Hyperparameter — A setting chosen before training (e.g., number of clusters K, learning rate) as opposed to a value learned from data.

Inference — Deriving new facts/conclusions from known facts and rules (in logic), or making a prediction with a trained model (in ML).

Loss Function — A measure of how wrong a model's predictions are; training tries to minimize this.

Overfitting — When a model memorizes training data (including its noise) instead of learning generalizable patterns, and performs poorly on new data.

Regression — Predicting a continuous numeric value (e.g., price, temperature).

Reinforcement Learning — Learning through trial-and-error interaction with an environment, guided by rewards/penalties.

Supervised Learning — Learning from labeled input-output pairs.

Unsupervised Learning — Learning structure from unlabeled data.

Chapter 9

Suggested Next Steps

1. Re-implement every code example from memory, without looking, then compare to the original.
2. Swap in a real dataset (e.g., from Kaggle) for the toy examples in Modules 3–5.
3. Read the original scikit-learn documentation for `DecisionTreeClassifier`, `KMeans`, and `MultinomialNB` to see the full range of hyperparameters not covered here.
4. Build one end-to-end mini-project: pick a real dataset, clean it, try 2–3 algorithms from this course, and compare their accuracy.